REȚELE DE CALCULATOARE · SEMESTRUL 1 · CURSUL 1

Introducere: ce este o rețea și cum este alcătuit Internetul

Pachetul didactic al săptămânii 1: note de curs, seminar, laborator Linux și laborator Windows (câte două ore fiecare)

Conf.univ.dr. Popescu-Bodorin Nicolaie, 2026
+40722882471, bodorin(on)live.com, https://lmrec.org/bodorin/

Convenție de terminologie

La prima apariție în fiecare parte a documentului, termenii de specialitate traduși în română sunt urmați, în paranteză, de termenul consacrat în limba engleză sau de forma „romgleză” folosită curent de practicieni — de exemplu, **comutator de pachete** (*packet switch*), **memorie tampon** (*buffer*, în romgleză „buffer”). Comenzile, numele câmpurilor din Wireshark și rezultatele afișate de programe sunt lăsate în engleză, așa cum apar pe ecran.

Partea	Conținut	Durată
A	Cursul 1 — Introducere: ce este o rețea și cum este alcătuit Internetul	2 h
B	Seminarul 1 — Raționamente cantitative despre rețele	2 h
C	Laboratorul 1 (Linux) — Explorarea rețelei din linia de comandă	2 h
D	Laboratorul 1 (Windows) — Explorarea rețelei cu PowerShell	2 h

Partea A · Cursul 1

După acest curs ar trebui să puteți

- să spuneți ce sunt o rețea de calculatoare, o gazdă, o legătură, un comutator de pachete și un protocol;
- să descrieți părțile Internetului: periferia, rețelele de acces și nucleul;
- să explicați comutarea de pachete și comutarea de circuite și să argumentați de ce Internetul o folosește pe prima;
- să descrieți cum sunt unite mii de rețele independente într-un singur Internet;
- să calculați întârzierea de transmisie și cea de propagare pe un traseu simplu;
- să numiți cele cinci niveluri ale stivei de protocoale Internet și să spuneți ce face fiecare.

Planul celor două ore

Partea	Subiect	Timp
Ora 1		
1	De ce studiem rețelele; cum funcționează acest curs	10 min
2	Ce este o rețea de calculatoare? Ce este Internetul?	15 min

Partea	Subiect	Timp
3	Ce este un protocol?	10 min
4	Periferia rețelei: gazde, rețele de acces, medii fizice	15 min
	Pauză	10 min
Ora 2		
5	Nucleul rețelei: comutarea de pachete și comutarea de circuite	20 min
6	O rețea de rețele	10 min
7	O primă privire asupra întârzierii	8 min
8	Organizarea pe niveluri, într-o singură imagine	7 min
9	Scurt istoric; rezumat	5 min

1. De ce studiem rețelele și cum funcționează acest curs

Aproape orice program pe care îl veți scrie în carieră comunică cu un alt program, aflat pe o altă mașină. O aplicație mobilă apelează un server; serverul apelează o bază de date și alte trei servicii; fiecare dintre acestea este replicat în mai multe centre de date. Când un astfel de sistem este lent, sau cade doar uneori, sau merge în laborator și nu merge la client, cauza se află foarte des în rețeaua dintre componente. Un inginer care înțelege rețeaua poate găsi cauza. Unul care o tratează ca pe o cutie neagră poate doar să repornească lucrurile și să spere.

Există și un al doilea motiv. Internetul este cel mai mare sistem ingineresc construit de oameni. Leagă miliarde de dispozitive, nu aparține nimănui, nu are control central și a continuat să funcționeze în timp ce a crescut cu multe ordine de mărime și a ajuns să transporte aplicații pe care proiectanții lui nu și le-au imaginat. Ideile care au făcut posibil acest lucru — organizarea pe niveluri (*layering*), comutarea de pachete (*packet switching*), livrarea fără garanții (*best effort*), plasarea inteligenței la margini (*at the edges*) — merită cunoscute pentru ele însele și reapar peste tot în informatică.

Organizarea cursului

- Cursul se întinde pe două semestre a câte 14 săptămâni, cu un curs de două ore și un laborator de două ore în fiecare săptămână.
- **Semestrul 1** coboară prin stiva de protocoale Internet, de sus în jos: aplicații, apoi transport, apoi nivelul rețea și rutarea. **Semestrul 2** acoperă nivelurile legătură de date și fizic, rețelele fără fir și mobile, securitatea și teme moderne precum rețelele definite prin software și rețelele din centrele de date.
- La laborator scrieți programe de rețea și desfaceți rețele în bucăți. Veți folosi Linux, C și Python și instrumente precum Wireshark. Prezența la laborator este obligatorie.
- Manualul principal este Kurose și Ross, *Computer Networking: A Top-Down Approach* (ediția a 8-a). Peterson și Davie, *Computer Networks: A Systems Approach*, este disponibil gratuit online și oferă o a doua perspectivă, utilă.
- Evaluarea, în fiecare semestru: examen scris 50 %, laborator 30 %, teme 20 %. Este nevoie de cel puțin nota 5 atât la examen, cât și la laborator.

2. Ce este o rețea de calculatoare?

O **rețea de calculatoare** este o mulțime de dispozitive de calcul conectate astfel încât să poată schimba date între ele. Definiția este largă în mod intenționat. Ea acoperă două laptopuri legate printr-un cablu, rețeaua Wi-Fi din această clădire, rețeaua națională a unui operator de telefonie mobilă și Internetul însuși.

Internetul este o rețea anume: o rețea mondială de rețele, în care rețele aflate în proprietăți diferite acceptă să folosească un set comun de protocoale — familia TCP/IP — astfel încât orice dispozitiv din oricare dintre ele să poată ajunge la orice dispozitiv din oricare alta. Îl putem descrie în două feluri.

2.1 Perspectiva „componentelor”: din ce este făcut

- **Gazde** (*hosts*), numite și **sisteme terminale** (*end systems*). Sunt dispozitivele de la margine, care rulează aplicații: laptopuri, telefoane, servere și, de acum, și televizoare, automobile, senzori și încuietori. „Gazde”, pentru că găzduiesc programele de aplicație.
- **Legături de comunicație** (*links*). Ele transportă biți între dispozitive: cablu de cupru, fibră optică, radio. **Rata de transmisie** (*transmission rate*) a unei legături (informal, lățimea ei de bandă — *bandwidth*) se măsoară în biți pe secundă.
- **Comutatoare de pachete** (*packet switches*). Gazda care transmite își împarte datele în bucăți, adaugă fiecăreia un antet și trimite **pachetele** (*packets*) astfel obținute. Comutatoarele de pachete primesc pachete pe o legătură și le retransmit pe alta. Cele două tipuri principale sunt **ruterele** (*routers*), folosite în nucleul rețelei, și **switch-urile** (comutatoare de nivel legătură de date — *link-layer switches*), folosite în interiorul rețelelor locale.
- **Furnizori de servicii Internet** (*Internet Service Providers*, ISP). Fiecare ISP este el însuși o rețea de comutatoare de pachete și legături. Gazdele ajung în Internet printr-un ISP: un furnizor rezidențial, un operator mobil, o universitate. În România, universitățile sunt conectate prin RoEduNet, rețeaua națională pentru educație și cercetare.
- **Protocoale**. Gazdele și comutatoarele rulează protocoale care controlează modul în care informația este trimisă și primită. Cele mai importante două sunt **IP** (Internet Protocol) și **TCP** (Transmission Control Protocol).
- **Standarde**. Protocoalele funcționează numai dacă toată lumea le implementează la fel. Standardele Internetului sunt elaborate de Internet Engineering Task Force (IETF) și publicate ca **RFC**-uri (Requests for Comments). Standardele de nivel legătură de date, precum Ethernet și Wi-Fi, provin de la comitetele IEEE 802.

Succesiunea de legături și comutatoare pe care o parcurge un pachet de la gazda sursă la gazda destinație se numește **rută** (*route*) sau **cale** (*path*).

2.2 Perspectiva serviciului: ce face pentru aplicații

Din punctul de vedere al programatorului, Internetul este o infrastructură care livrează date între programe de aplicație care rulează pe gazde diferite. Navigarea pe Web, mesageria, transmisia video și jocurile online sunt **aplicații distribuite** (*distributed applications*): constau din mai multe programe, pe mai multe gazde, care schimbă date. De reținut că aplicațiile rulează pe gazde, nu pe rutere. Ruterele mută pachete și nu știu nimic despre Web sau despre video.

Un program cere Internetului să livreze date printr-o interfață de programare numită **interfața socket** (*socket interface*). O puteți privi ca pe regulile poștei: pentru a trimite o scrisoare trebuie să o puneți într-un plic, să scrieți adresa într-un format fix și să o puneți într-o cutie poștală. Poșta oferă

apoi mai multe servicii (simplu, recomandat, expres). La fel, Internetul oferă mai multe servicii de livrare, iar alegerea celui potrivit pentru o aplicație este o decizie de proiectare la care vom reveni. Veți începe să programați cu socket-uri la laborator în săptămâna 3.

2.3 Clasificarea rețelelor

Două clasificări apar peste tot în literatură și în fișele de post, așa că vocabularul trebuie cunoscut.

După întinderea geografică	Întindere tipică	Exemplu
PAN – rețea personală (*personal area network*)	câțiva metri	telefon și căști prin Bluetooth
LAN – rețea locală (*local area network*)	o cameră, o clădire, un campus	rețeaua Ethernet și Wi-Fi a facultății
MAN – rețea metropolitană (*metropolitan area network*)	un oraș	rețeaua orășenească a unui operator de cablu sau fibră
WAN – rețea de arie largă (*wide area network*)	o țară, un continent	magistrala unui ISP; RoEduNet

După **topologie** (*topology*), adică forma în care sunt conectate nodurile (*nodes*): *magistrală* (*bus*: toate nodurile pe un singur cablu partajat, ca în primul Ethernet), *inel* (*ring*), *stea* (*star*: fiecare nod legat la un dispozitiv central, ca în Ethernetul comutat de azi și în Wi-Fi), *arbore* (*tree*) și *plasă* (*mesh*: mai multe căi alternative între noduri, ca în nucleul Internetului). Topologia hotărăște ce se întâmplă când ceva se defectează. Într-o stea, pierderea centrului îi deconectează pe toți; într-o plasă, traficul poate ocoli o legătură întreruptă.

După **rolul gazdelor**: în modelul **client–server** (*client–server*), un server mereu pornit așteaptă cereri de la clienți (așa funcționează Webul); în modelul **de la egal la egal** (*peer-to-peer*, în romgleză „P2P”), gazdele comunică direct și fiecare este și client, și server (așa funcționează BitTorrent).

3. Ce este un protocol?

Gândiți-vă cum întrebați un necunoscut cât este ceasul. Spuneți „Nu vă supărați…”. Așteptați. Dacă răspunde „Da?”, întrebați „Cât este ceasul?” și vă spune. Dacă vă ignoră sau răspunde într-o limbă pe care nu o cunoașteți, nu mai puneți întrebarea; renunțați sau încercați la altcineva. Există mesaje anume, ele vin într-o ordine anume, iar fiecare mesaj primit — sau neprimit — declanșează o acțiune anume. Acesta este un protocol.

Mașinile fac la fel, cu o deosebire: nimic nu poate fi lăsat pe seama bunului-simț. Totul trebuie specificat.

> **Definiție**
> Un **protocol** definește **formatul** și **ordinea** mesajelor schimbate între două sau mai multe entități care comunică, precum și **acțiunile** întreprinse la transmiterea sau la primirea unui mesaj ori la producerea unui alt eveniment (de exemplu, expirarea unui timp de așteptare).

Iată un exemplu real. Când tastați o adresă web, browserul stabilește mai întâi o conexiune TCP cu serverul (ea însăși un schimb de trei mesaje), apoi trimite o cerere în protocolul HTTP, iar serverul răspunde:

```
Browser → server:   GET /index.html HTTP/1.1
                    Host: www.example.org

Server → browser:   HTTP/1.1 200 OK
                    Content-Type: text/html
                    Content-Length: 1256

                    <!doctype html> ...
```

Fiecare element de aici este fixat de standardul HTTP: cuvântul GET, spațiile, șirul de versiune, sfârșiturile de linie, semnificația codului `200`. Un browser scris într-o țară și un server scris în alta, de oameni care nu s-au întâlnit niciodată, pot comunica deoarece ambii au urmat același document.

O mare parte din acest curs constă în studiul protocoalelor: **ce** problemă rezolvă fiecare, **cum** o rezolvă și **de ce** a fost proiectat așa și nu altfel.

> **Întrebare pentru sală**
> Ce ar trebui să facă browserul dacă nu sosește niciun răspuns? După cât timp? Să trimită din nou cererea? Dar dacă cererea era „transferă 100 de lei” și s-a pierdut răspunsul, nu cererea? Țineți minte întrebarea. Este problema centrală a nivelului transport și îi vom dedica trei cursuri.

4. Periferia rețelei

4.1 Gazde

Gazdele se împart, informal, în **clienți** și **servere**. Clienții sunt dispozitivele folosite direct de oameni. Serverele stochează și distribuie conținut — pagini web, video, e-mail — și astăzi cele mai multe se află în mari **centre de date** (*data centres*), clădiri care adăpostesc zeci sau sute de mii de mașini.

4.2 Rețele de acces

Rețeaua de acces (*access network*) este rețeaua care leagă o gazdă de primul ruter de pe calea ei, numit **ruter de margine** (*edge router*). Principalele tipuri sunt:

Tehnologie de acces	Cum funcționează	Rate tipice
DSL (linie digitală de abonat)	Folosește perechea telefonică existentă. Datele și vocea circulă pe același fir, la frecvențe diferite. Rata scade cu distanța față de centrală.	zeci de Mbit/s la descărcare, mai puțin la încărcare
Cablu (hibrid fibră–coaxial)	Folosește rețeaua de televiziune prin cablu. Fibra ajunge la un nod de cartier, iar cablul coaxial la locuințe. Coaxialul este un **mediu partajat** (*shared medium*): vecinii își împart capacitatea.	sute de Mbit/s până la circa 1 Gbit/s la descărcare
FTTH (fibră până la locuință)	Fibră optică până în locuință, de obicei ca rețea optică pasivă (*passive optical network*, PON), în care o fibră de la furnizor este împărțită între mai multe locuințe. Răspândită în orașele din România.	1 Gbit/s și peste
Ethernet	Cablu de cupru cu perechi torsadate (*twisted pair*) de la fiecare gazdă la un switch. Standardul în birouri, laboratoare și centre de date.	1–10 Gbit/s la birou; până la 400 Gbit/s între switch-uri
Wi-Fi (IEEE 802.11)	Gazdele schimbă semnale radio cu un punct de acces (*access point*) aflat la cel mult câteva zeci de metri; punctul de acces este legat prin cablu la restul rețelei.	de la zeci de Mbit/s la peste 1 Gbit/s

Tehnologie de acces	Cum funcționează	Rate tipice
Celular (4G, 5G)	Gazdele schimbă semnale radio cu o stație de bază (*base station*) care poate fi la kilometri distanță; rețeaua operatorului duce datele mai departe.	zeci până la sute de Mbit/s

Cifrele sunt ordine de mărime, nu promisiuni; se schimbă la câțiva ani. Ce nu se schimbă sunt două fapte de reținut. Legăturile de acces sunt adesea **asimetrice** (*asymmetric*), cu mai multă capacitate spre utilizator decât dinspre el. Și multe medii de acces sunt **partajate**, astfel încât rata obținută depinde de ce fac vecinii.

O rețea de locuință tipică le combină pe câteva dintre acestea: un terminal de fibră de la furnizor, un ruter care este totodată switch Ethernet și punct de acces Wi-Fi și vreo zece dispozitive în spatele lui.

4.3 Medii fizice

Un bit care călătorește de la o gazdă la alta traversează un șir de perechi emițător–receptor (*transmitter–receiver*). Între cele două capete ale fiecărei perechi, el se deplasează ca undă electromagnetică sau ca impuls de lumină printr-un **mediu fizic** (*physical medium*).

- **Mediile ghidate** (*guided media*) țin semnalul pe un traseu solid. *Cablul de cupru cu perechi torsadate* este ieftin și din el sunt făcute cablurile Ethernet. *Cablul coaxial* (*coaxial cable*) se folosește în rețelele de televiziune prin cablu. *Fibra optică* (*optical fibre*) transportă impulsuri de lumină, suportă rate uriașe pe distanțe mari cu foarte puține erori, este imună la interferențe electrice și este mediul tuturor legăturilor de mare distanță, inclusiv al celor de sub oceane.
- **Mediile neghidate** (*unguided media*) lasă semnalul să se răspândească prin aer sau prin spațiu: Wi-Fi, radio celular, legături prin satelit. Radioul nu are nevoie de cablu, dar semnalul slăbește cu distanța, este blocat și reflectat de obstacole și suferă interferențe de la alte emițătoare.

Vom studia nivelul fizic în semestrul 2. Deocamdată este de ajuns un singur fapt: în fibră și în cupru semnalele se propagă cu aproximativ două treimi din viteza luminii în vid, adică în jur de 2×10^8 m/s.

5. Nucleul rețelei

Nucleul rețelei (*network core*) este plasa de rutere și legături care interconectează rețelele de acces. Există două moduri fundamental diferite de a muta date printr-o asemenea plasă.

5.1 Comutarea de pachete

Gazda sursă împarte un mesaj lung în **pachete** mai mici. Fiecare pachet traversează legături și comutatoare de pachete până la destinație, independent de celelalte. Pe fiecare legătură, un pachet este transmis la **rata întreagă a acelei legături**.

Memorare și retransmitere (store-and-forward)

Cele mai multe comutatoare de pachete folosesc transmisia cu **memorare și retransmitere**: comutatorul trebuie să primească *întregul* pachet înainte de a putea începe să transmită primul lui bit pe legătura de ieșire.

Fie un pachet de L biți și o legătură cu rata de R biți pe secundă. Împingerea pachetului pe legătură durează L / R secunde. Cu un ruter între sursă și destinație există două legături, deci pachetul sosește

după 2*L*/*R* (ignorând, deocamdată, timpul în care semnalul parcurge distanța). În general, pe un traseu de *N* legături de aceeași rată:

$$d = N \cdot L / R$$

> **Exemplul rezolvat 1**
> Un pachet de 1500 de octeți are 12 000 de biți. Pe o legătură de 100 Mbit/s, L / R = 12 000 / 100 000 000 s = 0,12 ms.
> Pe două astfel de legături (un ruter între ele), întârzierea de memorare și retransmitere este 2 × 0,12 = 0,24 ms.

Cozi și pierderi

Fiecare comutator are, pentru fiecare legătură de ieșire, o **coadă de ieșire** (*output queue*) — o **memorie tampon** (*buffer*, în romgleză „buffer”). Dacă un pachet sosește și legătura este ocupată cu transmiterea altuia, pachetul nou așteaptă în coadă. Acest timp de așteptare este **întârzierea de așteptare în coadă** (*queueing delay*). Ea depinde de cât trafic sosește și de aceea variază de la un moment la altul.

Memoriile tampon sunt finite. Un pachet care sosește la o coadă plină nu are unde să stea, așa că el — sau unul aflat deja în coadă — este aruncat. Aceasta este **pierderea de pachete** (*packet loss*). În Internet, pierderea nu este o defecțiune excepțională. Este consecința normală a congestiei (*congestion*), iar protocoalele sunt proiectate să îi facă față.

Retransmitere (forwarding) și rutare

De unde știe un ruter pe ce legătură să trimită un pachet? Fiecare pachet poartă în antet (*header*) adresa destinației. Fiecare ruter are o **tabelă de dirijare** (*forwarding table*), care asociază adreselor de destinație (de fapt, prefixelor de adrese) legături de ieșire. Ruterul caută destinația și trimite pachetul pe legătura indicată de tabelă. Această acțiune locală este **retransmiterea** (*forwarding*).

Tabelele sunt completate de **protocoalele de rutare** (*routing protocols*), prin care ruterele schimbă informații și calculează căi bune prin întreaga rețea. Acest proces global este **rutarea** (*routing*). Analogia este cea a unui șofer fără hartă, care întreabă la fiecare intersecție „Pe unde o iau spre Cluj?” și de fiecare dată i se arată drumul următor. Retransmiterea înseamnă a urma indicatorul dintr-o intersecție. Rutarea este munca de a hotărî ce trebuie să scrie pe indicatoare.

5.2 Comutarea de circuite

Rețeaua telefonică tradițională funcționa altfel. Înainte de a circula vreo dată, rețeaua stabilește un **circuit** (*circuit*; de aici **comutarea de circuite**, *circuit switching*): rezervă capacitate pe fiecare legătură de pe traseu, pentru toată durata convorbirii. Odată circuitul stabilit, emițătorul poate transmite la o rată constantă, garantată, fără cozi și fără pierderi.

O legătură se împarte între circuite în unul din două moduri. La **multiplexarea cu divizare în frecvență** (*frequency-division multiplexing*, FDM), fiecare circuit primește permanent propria bandă de frecvență, ca posturile de radio. La **multiplexarea cu divizare în timp** (*time-division multiplexing*, TDM), timpul este tăiat în cadre (*frames*) și fiecare cadru în sloturi (*slots*), iar fiecare circuit primește același slot în fiecare cadru.

Exemplul rezolvat 2

O legătură de 2,048 Mbit/s folosește TDM cu 32 de sloturi, deci fiecare circuit primește 2 048 000 / 32 = 64 kbit/s.

Trimiterea unui fișier de 640 000 de biți pe un circuit durează 640 000 / 64 000 = 10 s. Dacă stabilirea circuitului durează 0,5 s, totalul este 10,5 s — indiferent dacă celelalte 31 de sloturi sunt folosite sau libere.

Ultima observație este punctul slab. Capacitatea rezervată și nefolosită este **irosită**: nimeni altcineva nu o poate folosi. Lucrul acesta este acceptabil pentru o convorbire telefonică, în care sunetul curge constant. Se potrivește prost calculatoarelor, al căror trafic vine **în rafale** (*bursty traffic*): cineva care citește o pagină web nu trimite nimic timp de treizeci de secunde și apoi, la un clic, vrea un megaoctet dintr-odată.

5.3 Comutare de pachete sau comutare de circuite?

La comutarea de pachete nu se rezervă nimic. Capacitatea este dată celui care are un pachet de trimis în acel moment. Această partajare la cerere se numește **multiplexare statistică** (*statistical multiplexing*). Un exemplu arată cât se câștigă.

Exemplul rezolvat 3: câți utilizatori pot împărți o legătură?

O legătură are rata de 1 Gbit/s. Fiecare utilizator, când este activ, transmite la 100 Mbit/s, dar este activ doar 10 % din timp.

Comutare de circuite. Fiecărui utilizator trebuie să i se rezerve permanent 100 Mbit/s. Legătura suportă 1000 / 100 = **10 utilizatori**.

Comutare de pachete. Admitem 35 de utilizatori. Probleme apar numai când 11 sau mai mulți sunt activi în același moment, pentru că atunci au nevoie de mai mult de 1 Gbit/s. Fiecare utilizator este activ independent, cu probabilitatea 0,1, deci numărul celor activi urmează o distribuție binomială și

P(cel puțin 11 activi din 35) = Σ (k = 11 … 35) C(35, k) × 0,1^k × 0,9^(35 − k) ≈ 0,0004.

Așadar, cu **35 de utilizatori** — de trei ori și jumătate mai mulți — cererea depășește capacitatea doar în circa 0,04 % din timp, iar atunci pachetele așteaptă puțin într-o coadă și nu se întâmplă nimic mai rău.

	Comutare de pachete	Comutare de circuite
Stabilire înainte de transmisie	nu	obligatorie
Capacitate	partajată la cerere	rezervată pe conexiune
Capacitate nefolosită	disponibilă celorlalți	irosită
Întârziere	variabilă; pot apărea cozi	constantă după stabilire
La suprasarcină	întârzierea crește, se pierd pachete	conexiunile noi sunt refuzate
Garanții	niciuna (*best effort*)	rată garantată
Potrivită pentru	trafic de date în rafale	trafic constant, cu rată fixă

Internetul folosește comutarea de pachete. Este mai simplă (ruterele nu păstrează stare pentru fiecare conexiune), mai ieftină și mult mai eficientă pentru traficul în rafale. Prețul este că **nu oferă**

nicio garanție: pachetele pot fi întârziate, pierdute sau pot sosi în altă ordine. Repararea acestor lucruri, acolo unde o aplicație are nevoie de ea, este treaba gazdelor, nu a rețelei. Această împărțire a sarcinilor este una dintre alegerile de proiectare definitorii ale Internetului și vom reveni la ea săptămâna viitoare, sub numele de *argumentul capăt-la-capăt* (*end-to-end argument*).

6. O rețea de rețele

Gazdele se conectează la ISP-uri de acces. Pentru ca oricare două gazde din lume să poată comunica, ISP-urile de acces trebuie, la rândul lor, interconectate. Cum?

Conectarea directă a fiecărui ISP de acces cu fiecare altul este imposibilă: cu sute de mii de rețele, numărul legăturilor ar crește cu pătratul acestui număr. Structura apărută în realitate a fost modelată de economie la fel de mult ca de inginerie și are mai multe niveluri.

- **Tranzit** (*transit*). Un ISP mic plătește unuia mai mare ca să îi ducă traficul către restul Internetului. ISP-ul mic este *clientul*, cel mare *furnizorul*. ISP-urile regionale cumpără tranzit de la cele naționale, iar acestea de la cele globale.
- **ISP-uri de nivel 1 (Tier-1).** În vârf se află aproximativ o duzină de rețele cu acoperire globală, care nu plătesc nimănui tranzit. Ele schimbă trafic între ele fără plată și astfel, împreună, pot ajunge la orice destinație.
- **Peering** (interconectare directă între egali; în romgleză „peering”). Două rețele care schimbă mult trafic se pot conecta direct și își pot duce reciproc traficul, de obicei fără plată. Amândouă economisesc taxa de tranzit pe care altfel ar plăti-o unui furnizor.
- **Puncte de interconectare (IXP – Internet exchange points).** Un IXP este o facilitate, în esență un switch mare, în care multe rețele se întâlnesc pentru a face peering. România are mai multe, de exemplu InterLAN și RoNIX, la București.
- **Conectare multiplă** (*multi-homing*, în romgleză „multihoming”). O rețea poate cumpăra tranzit de la doi sau mai mulți furnizori, ca să rămână conectată dacă unul dintre ei cade.
- **Rețele ale furnizorilor de conținut** (*content provider networks*). Companii precum Google, Meta, Netflix și rețelele de distribuție de conținut (CDN) operează rețele globale private, care le leagă centrele de date. Ele fac peering direct cu ISP-urile de acces și în IXP-uri, ocolind nivelurile superioare, pentru a aduce conținutul aproape de utilizatori, a-și reduce costurile și a controla performanța.

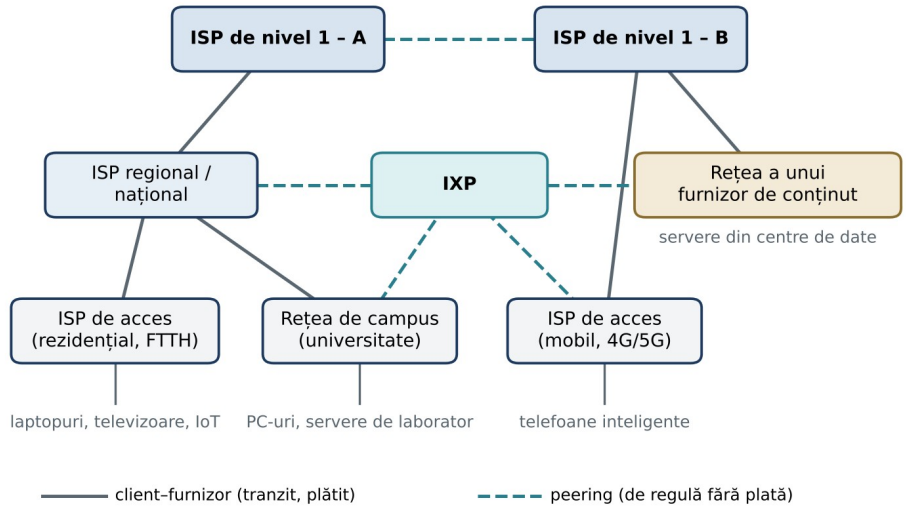


Figura 1. Internetul ca rețea de rețele. Liniile continue sunt legături plătite client–furnizor; liniile întrerupte sunt legături de peering.

Consecința de reținut: un pachet de la telefonul vostru către un server traversează de regulă mai multe rețele operate independent. Niciuna nu controlează întregul traseu și niciuna nu este obligată să acorde pachetului vostru un tratament anume. Veți vedea acest lucru chiar voi, la primul laborator, cu instrumentul `traceroute`.

7. O primă privire asupra întârzierii

În drumul de la un nod la următorul, un pachet este întârziat din patru motive diferite.

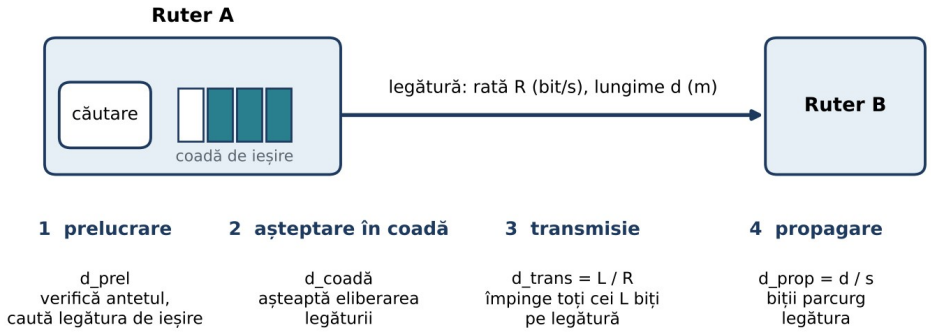


Figura 2. Cele patru surse de întârziere pe un salt.

1. **Întârzierea de prelucrare** (*processing delay*). Ruterul examinează antetul, verifică erorile de bit și hotărăște legătura de ieșire. Pe ruterele moderne durează microsecunde.
2. **Întârzierea de așteptare în coadă** (*queueing delay*). Pachetul așteaptă eliberarea legăturii. De la zero la multe milisecunde, în funcție de congestie.
3. **Întârzierea de transmisie** (*transmission delay*), L / R. Timpul necesar pentru a împinge toți cei L biți ai pachetului pe o legătură de rată R. Depinde de lungimea pachetului și de rata legăturii și **nu** depinde de lungimea legăturii.
4. **Întârzierea de propagare** (*propagation delay*), d / s. Timpul în care un bit parcurge o legătură de lungime d cu viteza semnalului s (circa 2×10^8 m/s). Depinde de distanță și **nu** depinde de lungimea pachetului sau de rata legăturii.

Întârzierea totală într-un nod este suma lor: **d_nod = d_prel + d_coadă + d_trans + d_prop**.

Studenții confundă întârzierea de transmisie cu cea de propagare mai des decât orice altceva din acest capitol. O coloană de zece mașini pe autostradă lămurește deosebirea. Fiecare mașină are nevoie de 12 secunde ca să treacă printr-un punct de taxare, deci întreaga coloană trece în 2 minute: aceasta este întârzierea de transmisie și depinde de numărul de mașini și de viteza punctului de taxare. Apoi mașinile parcurg 100 km cu 100 km/h, ceea ce durează o oră: aceasta este întârzierea de propagare și depinde de distanță și de viteza de deplasare. Un punct de taxare mai rapid nu scurtează drumul. Un drum mai scurt nu grăbește taxarea.

> **Exemplul rezolvat 4**
> Un pachet de 1500 de octeți (12 000 de biți) este trimis de la Cluj-Napoca la București pe un traseu de fibră de aproximativ 450 km, pe o legătură de 100 Mbit/s.
> Întârzierea de transmisie: 12 000 / 100 000 000 s = 0,12 ms.
> Întârzierea de propagare: 450 000 m / 200 000 000 m/s = 2,25 ms.
> Aici domină propagarea, de aproape 20 de ori. Pe o legătură lentă este invers: la 1 Mbit/s, întârzierea de transmisie a aceluiași pachet ar fi de 12 ms.

Înrudit cu întârzierea este **debitul efectiv** (*throughput*): rata, în biți pe secundă, cu care datele ajung efectiv la receptor. De-a lungul unui traseu, debitul este limitat de cea mai lentă legătură, **legătura îngustă** (*bottleneck*). Un fișier de 8 megaocteți are 64 de milioane de biți; dacă cea mai lentă legătură de pe traseu vă oferă 2 Mbit/s, transferul durează cel puțin 64 / 2 = 32 de secunde, oricât de rapide ar fi celelalte legături. Săptămâna viitoare tratăm mai în profunzime întârzierea, pierderile și debitul.

8. Organizarea pe niveluri, într-o singură imagine

Internetul are de rezolvat multe probleme deodată: transformarea biților în semnale, partajarea unui canal radio, găsirea unui drum prin lume, recuperarea pachetelor pierdute, atribuirea unui înțeles datelor. Rezolvarea lor într-un singur bloc monolitic de software ar fi de negestionat. Soluția este **organizarea pe niveluri** (*layering*). Munca este împărțită în niveluri, fiecare oferind un serviciu nivelului de deasupra și folosind serviciul nivelului de dedesubt, fără ca vreunul să trebuiască să știe cum este implementat celălalt.

Avantajul este că un nivel poate fi schimbat fără a le atinge pe celelalte. Browserul funcționează neschimbat peste Ethernet, Wi-Fi sau 5G, iar Wi-Fi nu a trebuit reproiectat când a fost inventată transmisia video.

Nivel	Rolul său	Exemple	Unitatea de date
5 Aplicație (*application*)	Dă înțeles schimbului dintre programele de aplicație.	HTTP, DNS, SMTP	mesaj (*message*)
4 Transport (*transport*)	Livrează date între *procese* de pe două gazde; poate adăuga fiabilitate (*reliability*) și controlul congestiei (*congestion control*).	TCP, UDP, QUIC	segment
3 Rețea (*network*)	Mută pachete între *gazde*, prin multe rețele; adresare și rutare.	IP, ICMP; OSPF, BGP	datagramă (*datagram*)
2 Legătură de date (*link*)	Mută un pachet peste *o singură legătură*, până la nodul vecin.	Ethernet, Wi-Fi	cadru (*frame*)

Nivel	Rolul său	Exemple	Unitatea de date
1 Fizic (*physical*)	Mută biți individuali prin mediu.	fibră, cupru, semnalizare radio	bit

Pe măsură ce datele coboară prin stivă la emițător, fiecare nivel își adaugă propriul **antet** (*header*) în fața a ceea ce a primit de la nivelul de deasupra. Aceasta este **încapsularea** (*encapsulation*). La receptor, fiecare nivel își citește și își înlătură antetul și trimite restul în sus.

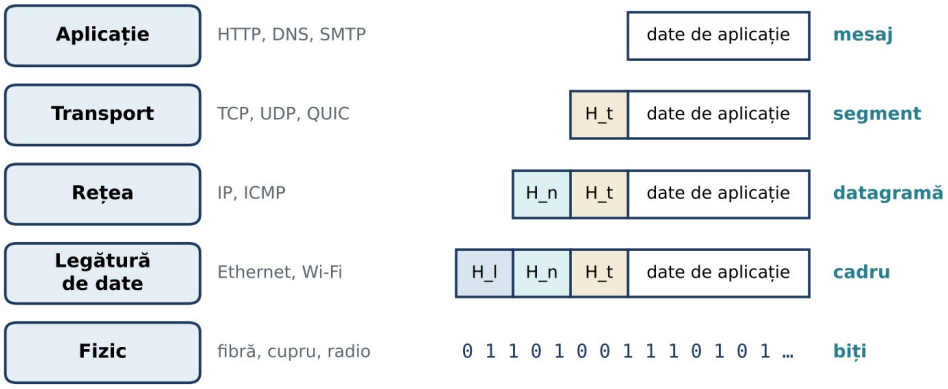


Figura 3. Încapsularea: fiecare nivel adaugă un antet (H_t transport, H_n rețea, H_l legătură de date) la datele primite de la nivelul de deasupra.

Gazdele implementează toate cele cinci niveluri ale stivei de protocoale (*protocol stack*). Ruterele le implementează doar pe cele trei de jos, iar switch-urile pe cele două de jos. Acesta este sensul precis în care nucleul Internetului este simplu, iar inteligența lui stă la margini. Săptămâna viitoare comparăm acest model cu cinci niveluri cu modelul de referință OSI (*OSI reference model*), cu șapte niveluri.

9. Scurt istoric

Anul	Eveniment
1961–64	Leonard Kleinrock, Paul Baran și Donald Davies dezvoltă, independent, ideea comutării de pachete.
1969	Sunt conectate primele patru noduri ale ARPANET: UCLA, Stanford Research Institute, UC Santa Barbara și Universitatea din Utah.
1971–72	Ray Tomlinson trimite primul e-mail prin rețea; ARPANET este demonstrat public.
1974	Vinton Cerf și Robert Kahn publică proiectul TCP și al unei arhitecturi de interconectare a unor rețele diferite.
1983	La 1 ianuarie, ARPANET trece la TCP/IP. În același an este specificat sistemul numelor de domenii (DNS).
1986–88	NSFNET leagă universitățile americane. După o serie de colapsuri de congestie, Van Jacobson adaugă în TCP controlul congestiei.
1989–91	Tim Berners-Lee inventează World Wide Web la CERN: HTML, HTTP, primul server și primul browser.
1993	Apare browserul grafic Mosaic. Este înregistrat domeniul de nivel superior `.ro`.

Anul	Eveniment
1995	NSFNET este retras; furnizorii comerciali preiau magistrala Internetului.
1998	Este specificat IPv6 (RFC 2460).
2007–10	Telefoanele inteligente și banda largă mobilă aduc Internetul în buzunarul fiecăruia; marile rețele de conținut remodelează topologia.
2021–22	Sunt standardizate QUIC (RFC 9000) și HTTP/3 (RFC 9114) — un transport nou, după patruzeci de ani de TCP.

Două lucruri ies în evidență. Arhitectura de bază din 1974–83 este cea folosită și astăzi. Și aproape toate aplicațiile care contează azi au fost inventate mai târziu, de oameni care nu au avut nevoie de permisiunea rețelei pentru a le lansa.

Rezumat

- O rețea leagă **gazde** prin **legături** și **comutatoare de pachete**; Internetul este o rețea de rețele care folosesc în comun protocoalele TCP/IP.
- Un **protocol** fixează formatul și ordinea mesajelor și acțiunile întreprinse la transmitere, la recepție și la alte evenimente.
- Gazdele ajung în Internet prin **rețele de acces** (DSL, cablu, fibră, Ethernet, Wi-Fi, celular), peste **medii fizice** ghidate sau neghidate.
- Nucleul folosește **comutarea de pachete**, cu rutere care memorează și retransmit. Când legăturile sunt ocupate, pachetele așteaptă în cozi și se pot pierde. **Comutarea de circuite** rezervă capacitate și oferă garanții, dar irosește capacitate la trafic în rafale.
- **Retransmiterea** (*forwarding*) este acțiunea locală a unui ruter de a muta un pachet pe legătura de ieșire potrivită; **rutarea** este calculul, la nivelul întregii rețele, al tabelelor care fac acest lucru posibil.
- ISP-urile se interconectează prin **tranzit** și **peering**, adesea în **IXP**-uri; furnizorii de conținut își operează propriile rețele globale.
- Întârzierea are patru componente: prelucrare, așteptare în coadă, **transmisie (L/R)** și **propagare (d/s)**. Debitul efectiv este limitat de legătura îngustă.
- Stiva de protocoale are cinci **niveluri**: aplicație, transport, rețea, legătură de date, fizic. Fiecare își adaugă antetul: **încapsulare**.

Termeni-cheie

gazdă (*host*) / sistem terminal (*end system*) · legătură (*link*) · rată de transmisie (*transmission rate*) · pachet (*packet*) · ruter (*router*) · switch (*link-layer switch*) · ISP · protocol · RFC · socket · client–server · de la egal la egal (*peer-to-peer*) · LAN / WAN · topologie (*topology*) · rețea de acces (*access network*) · ruter de margine (*edge router*) · medii ghidate / neghidate (*guided / unguided media*) · comutare de pachete (*packet switching*) · memorare și retransmitere (*store-and-forward*) · întârziere de așteptare în coadă (*queueing delay*) · pierdere de pachete (*packet loss*) · tabelă de dirijare (*forwarding table*) · protocol de rutare (*routing protocol*) · comutare de circuite (*circuit switching*) · FDM / TDM · multiplexare statistică (*statistical multiplexing*) · tranzit (*transit*) · peering · IXP · conectare multiplă (*multi-homing*) · întârziere de transmisie (*transmission delay*) · întârziere de propagare (*propagation*

delay) · debit efectiv (*throughput*) · legătură îngustă (*bottleneck*) · nivel (*layer*) · încapsulare (*encapsulation*)

Întrebări recapitulative

1. Care este deosebirea dintre o gazdă și un comutator de pachete? Este un ruter Wi-Fi de acasă o gazdă?
2. Enunțați cele trei lucruri pe care le definește un protocol. Identificați-le pe fiecare în exemplul HTTP din secțiunea 3.
3. De ce sunt adesea asimetrice legăturile de acces rezidențiale?
4. Un ruter a primit jumătate dintr-un pachet. De ce nu poate începe încă să îl retransmită?
5. Dați exemplu de o aplicație pentru care comutarea de circuite s-ar potrivi mai bine decât comutarea de pachete și spuneți de ce.
6. Explicați cu propriile cuvinte deosebirea dintre retransmitere (*forwarding*) și rutare.
7. De ce ar accepta două ISP-uri să își transporte reciproc traficul gratuit?
8. Ce componentă a întârzierii se schimbă dacă înlocuiți o legătură de 100 Mbit/s cu una de 1 Gbit/s de aceeași lungime? Care rămâne la fel?
9. Ce niveluri implementează un ruter? De ce nu are nevoie de celelalte?

Exerciții

1. Cinci pachete a câte 1500 de octeți sunt trimise unul după altul pe un traseu de trei legături, fiecare de 10 Mbit/s, prin două rutere cu memorare și retransmitere. Ignorând propagarea, prelucrarea și cozile, când sosește ultimul bit al ultimului pachet?
2. Un satelit geostaționar se află la circa 36 000 km deasupra Pământului. Care este întârzierea de propagare de la o stație de sol până la satelit și înapoi la o altă stație de sol? (Undele radio se propagă cu 3×10^8 m/s.) Ce înseamnă aceasta pentru o convorbire telefonică?
3. O legătură de 1 Gbit/s este împărțită de utilizatori care au nevoie fiecare de 100 Mbit/s când sunt activi și sunt activi 10 % din timp. Cu comutare de pachete și 35 de utilizatori, care este probabilitatea ca *toți cei 35* să fie activi simultan? Comparați-o cu probabilitatea găsită în exemplul rezolvat 3 și comentați.
4. Trebuie să mutați 40 de teraocteți de date de la Iași la București. Aveți o legătură dedicată de 100 Mbit/s. Un curier poate livra discurile în 8 ore. Ce variantă este mai rapidă?
5. Cât de lung este un pachet, în metri? Considerați un pachet de 12 000 de biți pe o legătură de fibră de 100 Mbit/s. (Indicație: cât de departe ajunge primul bit înainte ca ultimul bit să plece de la emițător?)

Răspunsuri

1. $L / R = 12\,000 / 10\,000\,000$ s = 1,2 ms. Primul pachet sosește după $3 \times 1{,}2 = 3{,}6$ ms; fiecare pachet următor sosește la 1,2 ms după cel dinainte. Al cincilea sosește la $(3 + 4) \times 1{,}2 =$ **8,4 ms**. În general, P pachete pe N legături durează $(N + P - 1) \cdot L / R$.
2. Urcare: $36\,000\,000 / 300\,000\,000 = 0{,}12$ s. Urcare și coborâre: **0,24 s**. O întrebare și răspunsul ei sunt despărțite de circa o jumătate de secundă de întârziere pură de propagare, pe care oamenii o observă și care face conversația stânjenitoare. Sateliții de pe orbită joasă, aflați la câteva sute de kilometri, evită acest neajuns.

3. $0{,}1^{35} = 10^{-35}$, adică practic niciodată. Riscul multiplexării statistice nu vine din faptul că toți ar fi activi împreună. Vine din faptul că pot fi activi cu puțin mai mulți utilizatori decât poate duce legătura, și de aceea cifra relevantă este 0,0004.
4. 40 TB = 40×10^{12} octeți = $3{,}2 \times 10^{14}$ biți. La 10^8 bit/s, aceasta durează $3{,}2 \times 10^6$ s, adică circa **37 de zile**. Curierul câștigă detașat. Debitul și întârzierea sunt măsuri diferite: curierul are un debit enorm și o întârziere îngrozitoare.
5. Transmisia pachetului durează 0,12 ms. În acest timp, primul bit parcurge 0,00012 s × 2×10^8 m/s = **24 km**. Pachetul este întins pe 24 km de fibră.

Bibliografie și pașii următori

Obligatoriu

- Kurose și Ross, *Computer Networking: A Top-Down Approach*, ediția a 8-a, capitolul 1: secțiunile despre ce este Internetul, periferia rețelei, nucleul rețelei și istoria rețelelor.

Recomandat

- Peterson și Davie, *Computer Networks: A Systems Approach*, capitolul 1 (gratuit la book.systemsapproach.org).
- Tanenbaum, Feamster și Wetherall, *Computer Networks*, ediția a 6-a, capitolul 1 (edițiile anterioare au apărut și în limba română, cu titlul *Rețele de calculatoare*).
- D. D. Clark, „The Design Philosophy of the DARPA Internet Protocols”, *Proc. ACM SIGCOMM*, 1988. Zece pagini despre de ce este construit Internetul așa cum este, scrise de unul dintre arhitecții lui.

Laboratorul 1 (săptămâna aceasta)

Pregătirea mediului de lucru. Veți afla adresele și poarta implicită ale mașinii voastre, veți măsura timpi dus-întors cu `ping` și veți folosi `traceroute` pentru a enumera ruterele dintre laborator și mai multe servere din România și din străinătate. Veniți cu răspunsul la două întrebări: câte rețele traversează pachetul vostru și de unde provine cea mai mare parte a întârzierii?

Cursul următor

Arhitecturi pe niveluri, în detaliu: modelele de referință OSI și TCP/IP, încapsularea pas cu pas și o tratare cantitativă a întârzierii, a pierderilor și a debitului.

PARTEA B · SĂPTĂMÂNA 1

Seminarul 1: Raționamente cantitative despre rețele

Ședință de probleme de două ore (2 × 50 de minute), asociată Cursului 1

Seminarul transformă ideile din curs în calcule și în decizii de proiectare (*design decisions*). Studenții lucrează în perechi; după fiecare problemă, o pereche își prezintă rezolvarea la tablă. Nu este nevoie de calculatoare, doar de un calculator de buzunar.

La sfârșitul seminarului ar trebui să puteți

- să convertiți corect între octeți (*bytes*) și biți (*bits*) și între unitățile uzuale de rată (*rate*);
- să calculați întârzierea de transmisie (*transmission delay*), de propagare (*propagation delay*) și cea capăt-la-capăt (*end-to-end delay*) pe un traseu cu mai multe salturi (*hops*);
- să comparați cantitativ comutarea de mesaje (*message switching*), comutarea de pachete (*packet switching*) și comutarea de circuite (*circuit switching*);
- să explicați printr-un calcul de ce funcționează multiplexarea statistică (*statistical multiplexing*) și când dă greș;
- să specificați complet un protocol mic: mesaje, ordine, acțiuni, erori (*messages, order, actions, errors*).

Plan

Pas	Activitate	Timp
A	Încălzire: unități și ordine de mărime	10 min
B	Problemele 1–2: întârzierea pe o legătură și pe un traseu	25 min
C	Problema 3: de ce pachete? Segmentarea (*segmentation*) și prețul ei	15 min
	Pauză	10 min
D	Problemele 4–5: partajarea unei legături; cozi	25 min
E	Problema 6: proiectați un protocol (lucru în grup)	20 min
F	Problemele 7–8: costul antetelor (*header overhead*); citirea unui traceroute	15 min
	Încheiere și temă	(în F)

A. Încălzire: unități

Stabiliți convențiile folosite tot anul, pentru că cele mai multe răspunsuri greșite din acest capitol sunt erori de unități.

- 1 octet = 8 biți. Dimensiunile fișierelor se dau în **octeți** (B); ratele legăturilor, în **biți pe secundă** (bit/s). Convertiți întotdeauna în biți înainte de a împărți.
- Pentru rate, prefixele sunt zecimale (*decimal prefixes*): 1 kbit/s = 10^3 bit/s, 1 Mbit/s = 10^6 bit/s, 1 Gbit/s = 10^9 bit/s.

- În acest curs și dimensiunile fișierelor sunt zecimale (1 MB = 10^6 octeți), dacă nu se precizează altfel. Sistemele de operare raportează adesea unități binare (*binary units*: 1 MiB = 2^{20} octeți); diferența este de circa 5 % la nivel de megaoctet.
- Viteza semnalului (*signal speed*) în cupru și în fibră: $s \approx 2 \times 10^8$ m/s = 200 km pe milisecundă. În aer și în vid: 3×10^8 m/s.

Întrebări rapide (răspuns oral). (a) Câți biți are un pachet de 1500 de octeți? (b) Cât durează transferul unui fișier de 5 MB la 20 Mbit/s? (c) Ce distanță parcurge un semnal în fibră într-o milisecundă? (d) Un telefon afișează „descărcare: 4 MB/s”. Cât înseamnă în Mbit/s?

> **Rezolvare**
> (a) 12 000 de biți. (b) $40 \times 10^6 / 20 \times 10^6 = 2$ s. (c) 200 km. (d) 32 Mbit/s.

B. Întârzierea

Problema 1 — o singură legătură

Gazdele A și B sunt unite printr-o singură legătură cu rata R = 10 Mbit/s și lungimea de 2000 km. Viteza semnalului este 2×10^8 m/s. A trimite un pachet de 1500 de octeți.

1. Calculați întârzierea de transmisie și întârzierea de propagare.
2. Când ajunge ultimul bit la B?
3. În clipa în care A termină de transmis, unde se află primul bit?
4. Pentru ce lungime a legăturii ar fi egale cele două întârzieri?
5. Cât de lung este, în metri, un bit pe această legătură?

> **Rezolvare**
> 1. $d_{trans} = 12\,000 / 10^7$ s = 1,2 ms. $d_{prop} = 2 \times 10^6$ m / 2×10^8 m/s = 10 ms.
> 2. Ultimul bit pleacă la 1,2 ms și călătorește 10 ms: ajunge la 11,2 ms.
> 3. A călătorit 1,2 ms: 1,2 ms × 200 km/ms = 240 km de la A. Întregul pachet ocupă 240 km din legătură.
> 4. $d / s = L / R$ dă $d = s \cdot L / R = 240$ km.
> 5. Un bit durează $1 / R = 0{,}1$ µs și ocupă deci $s / R = 20$ m.

Problema 2 — un traseu cu trei legături

Un pachet de 12 000 de biți merge de la gazda (*host*) A, prin ruterele (*routers*) R1 și R2, la gazda B. Fiecare ruter are nevoie de 20 µs pentru prelucrare (*processing*); cozile (*queues*) sunt goale.

Legătura	Rata	Lungimea
A – R1 (LAN de campus)	100 Mbit/s	1 km
R1 – R2 (legătură regională)	10 Mbit/s	500 km
R2 – B (legătură de centru de date)	1 Gbit/s	50 km

1. Calculați întârzierea capăt-la-capăt a pachetului.
2. Ce componentă domină și pe ce legătură?
3. Care este debitul (*throughput*) maxim al unui transfer lung de fișier de la A la B? Cât durează un fișier de 30 MB?
4. Operatorul modernizează prima legătură la 1 Gbit/s. Cu cât se accelerează transferul fișierului?

Rezolvare

1. Transmisie: 0,12 + 1,2 + 0,012 = 1,332 ms. Propagare: 0,005 + 2,5 + 0,25 = 2,755 ms. Prelucrare: 2 × 0,02 = 0,04 ms. Total ≈ 4,13 ms.
2. Propagarea pe legătura de 500 km (2,5 ms), urmată de transmisia pe aceeași legătură (1,2 ms). Legătura lentă și lungă le domină pe amândouă.
3. Legătura îngustă (*bottleneck*) este cea de 10 Mbit/s. 30 MB = 240 × 10^6 biți, deci cel puțin 24 s.
4. Deloc: legătura îngustă a rămas aceeași. Modernizarea unei legături care nu este cea îngustă nu crește debitul.

C. De ce pachete?

Problema 3 — segmentarea

Un mesaj de 1,2 × 10^6 biți trebuie să traverseze trei legături a câte 2 Mbit/s, prin două comutatoare cu memorare și retransmitere (*store-and-forward switches*). Ignorați propagarea și prelucrarea.

1. **Comutare de mesaje.** Mesajul este trimis întreg. Când sosește?
2. **Comutare de pachete.** Mesajul este tăiat în 100 de pachete a câte 12 000 de biți. Când sosește ultimul pachet?
3. Fiecare pachet are nevoie de un antet (*header*) de 40 de octeți. Refaceți punctul 2 și exprimați costul suplimentar în procente.
4. Dați încă două avantaje și un dezavantaj al împărțirii mesajelor în pachete.

Rezolvare

1. Fiecare salt durează 1,2 × 10^6 / 2 × 10^6 = 0,6 s. Trei salturi: 1,8 s.
2. L / R = 6 ms. Folosind (N + P − 1) · L / R cu N = 3 legături și P = 100 de pachete: 102 × 6 ms = 0,612 s — de aproape trei ori mai repede, pentru că cele trei legături lucrează în paralel, pe pachete diferite.
3. Pachetele au acum 12 320 de biți, L / R = 6,16 ms, total 102 × 6,16 = 0,628 s. Cost suplimentar: 320 / 12 320 ≈ 2,6 %.
4. Avantaje: o eroare de bit impune retransmiterea unui singur pachet mic, nu a întregului mesaj; un mesaj lung nu blochează un comutator pentru toți ceilalți. Dezavantaj: costul antetelor (*header overhead*), iar pachetele pot sosi în altă ordine (*out of order*) și trebuie reasamblate (*reassembly*).

D. Partajarea unei legături

Problema 4 — circuite sau pachete?

O legătură de 100 Mbit/s deservește utilizatori care transmit fiecare la 10 Mbit/s când sunt activi și sunt activi 20 % din timp, independent unul de altul.

1. Câți utilizatori poate suporta comutarea de circuite?
2. Cu comutare de pachete și 30 de utilizatori, care sunt numărul mediu de utilizatori activi și încărcarea medie a legăturii?
3. Scrieți expresia probabilității ca cei 30 de utilizatori să ceară, împreună, mai mult decât poate duce legătura. Valoarea ei este de aproximativ 0,026. Interpretați-o.
4. Ați admite 30 de utilizatori? Ce ați vrea să știți înainte de a hotărî?

Rezolvare

1. 100 / 10 = 10 utilizatori.
2. 30 × 0,2 = 6 utilizatori activi în medie, adică o încărcare de 60 Mbit/s: 60 % din capacitate.
3. P(mai mult de 10 activi) = Σ (k = 11 … 30) C(30, k) × 0,2^k × 0,8^(30 − k) ≈ 0,026. În circa 2,6 % din timp cererea depășește capacitatea și pachetele așteaptă în coadă.
4. Depinde de aplicație. Pentru navigarea pe Web, episoade scurte de așteptare în 2,6 % din timp sunt acceptabile, iar numărul clienților deserviți se triplează. Pentru un apel vocal sau pentru o legătură de chirurgie la distanță, s-ar putea să nu fie. Comparați cu exemplul din curs (35 de utilizatori activi 10 % din timp dădeau 0,0004): contează mai mult cât de „în rafale” transmit utilizatorii decât câți sunt.

Problema 5 — se formează o coadă

Legătura de ieșire (*outgoing link*) a unui ruter are rata de 1 Mbit/s. Toate pachetele au 12 000 de biți.

1. Cinci pachete sosesc în aceeași clipă la o coadă goală. Care este întârzierea de așteptare a fiecăruia și care este media?
2. Pachetele sosesc câte unul la fiecare 12 ms. Care este întârzierea de așteptare?
3. Pachetele sosesc câte unul la fiecare 10 ms. Ce se întâmplă? Dacă memoria tampon (*buffer*, în romgleză „buffer”) ține 20 de pachete, cam când este aruncat primul pachet?
4. Definiți raportul (rata sosirilor, în bit/s) / (rata legăturii) pentru cazurile 2 și 3. Ce înseamnă valoarea 1?

Rezolvare

1. L / R = 12 ms. Pachetele așteaptă 0, 12, 24, 36 și 48 ms; media este 24 ms. În general, al n-lea pachet așteaptă (n − 1) · L / R.
2. Zero. Fiecare pachet termină exact când sosește următorul.
3. Sosesc 100 de pachete/s, dar pot pleca doar 83,3 pachete/s, deci coada crește cu circa 16,7 pachete în fiecare secundă. O memorie tampon de 20 de pachete se umple în aproximativ 1,2 s; după aceea este aruncat cam un pachet din șase.
4. Cazul 2: 1,0. Cazul 3: 1,2. Acest raport este **intensitatea traficului** (*traffic intensity*). Peste 1, coada crește nelimitat, oricare ar fi mărimea memoriei tampon; o memorie mai mare doar amână pierderea și adaugă întârziere. Îl studiem săptămâna viitoare.

E. Proiectați un protocol

Problema 6 — lucru în grup

În grupuri de trei sau patru, proiectați un protocol de aplicație pentru un **serviciu de oră exactă**: un client întreabă un server cât este ceasul. Aveți 12 minute; apoi două grupuri prezintă, iar celelalte încearcă să le „strice” proiectul. Specificația trebuie să răspundă la fiecare întrebare de mai jos, pentru că un program nu poate ghici.

- Ce anume trimite clientul? Text sau binar? De unde știe serverul unde se termină mesajul?
- Ce anume răspunde serverul? În ce format și în ce fus orar este ora?
- Ce face serverul cu un mesaj pe care nu îl înțelege?
- Ce face clientul dacă nu vine niciun răspuns? După cât timp? De câte ori încearcă?
- Este dăunător dacă serverul primește aceeași cerere de două ori?

Un răspuns posibil

Mesajele sunt linii de text ASCII terminate cu CR LF. Cerere: `TIME?`. Răspuns: `TIME 2026-10-06T09:30:00Z` (ISO 8601, întotdeauna UTC). Răspuns de eroare: `ERR 400 unknown request`.

Ordinea. Clientul vorbește primul; serverul trimite exact un răspuns pentru fiecare cerere.

Acțiunile. Serverul răspunde cu `ERR 400` la orice linie diferită de `TIME?` și rămâne pregătit. Clientul așteaptă răspunsul 2 s, apoi repetă cererea, de cel mult trei ori, după care raportează utilizatorului eșecul.

Duplicatele sunt inofensive: a întreba de două ori cât este ceasul nu schimbă nimic pe server. O astfel de cerere se numește **idempotentă** (*idempotent*). O cerere de felul „transferă 100 de lei” nu este, iar repetarea ei orbește este periculoasă; are nevoie de un identificator de cerere (*request identifier*), ca serverul să poată recunoaște o repetare. Revenim la aceasta în cursurile despre nivelul transport (*transport layer*).

Idei de scos în evidență în discuție: fiecare grup a uitat ceva (de obicei regula de sfârșit de mesaj sau timpul de așteptare); RFC-urile reale sunt lungi exact din acest motiv; iar cele trei părți ale definiției din curs — format, ordine, acțiuni — dau o listă de verificare pentru orice protocol.

F. Niveluri și trasee

Problema 7 — costul antetelor

O aplicație trimite un mesaj de 100 de octeți peste TCP (antet de 20 de octeți), IPv4 (antet de 20 de octeți) și Ethernet (antet de 14 octeți și sfârșit de cadru — *trailer* — de 4 octeți).

1. Cât de lung este cadrul (*frame*) și ce fracțiune din el este cost suplimentar (*overhead*)?
2. Repetați pentru un mesaj de 1460 de octeți.
3. Ce antete citește un switch (comutator de nivel legătură de date)? Un ruter? Gazda destinație?

Rezolvare

1. 100 + 20 + 20 + 18 = 158 de octeți; cost suplimentar 58 / 158 ≈ 37 %.

2. 1518 octeți; cost suplimentar 58 / 1518 ≈ 3,8 %. Mesajele mici sunt scumpe, iar acesta este unul dintre motivele pentru care aplicațiile grupează datele.

3. Switch-ul citește doar antetul Ethernet. Ruterul citește antetul Ethernet și antetul IP. Gazda destinație le citește pe toate, iar aplicația ei citește mesajul.

Problema 8 — citirea unui traceroute

Rezultatul de mai jos este inventat pentru exercițiu. Fiecare linie este un ruter de pe traseu, cu trei măsurători ale timpului dus-întors (*round-trip time*, RTT).

```
1  gw.campus.example.edu (10.20.0.1)          0.4 ms   0.3 ms   0.4 ms
2  core1.campus.example.edu (10.0.0.1)        0.7 ms   0.6 ms   0.7 ms
3  cta-1.nren.example.net (192.0.2.17)        1.9 ms   1.8 ms   2.1 ms
4  buc-2.nren.example.net (192.0.2.41)        5.2 ms   5.1 ms   5.4 ms
5  * * *
6  vie-1.transit.example.net (198.51.100.9)  22.8 ms  23.1 ms  22.7 ms
7  fra-3.transit.example.net (198.51.100.33) 34.6 ms  34.2 ms  61.0 ms
8  nyc-1.transit.example.net (198.51.100.77) 118.3 ms 117.9 ms 118.6 ms
9  edge.cdn.example.com (203.0.113.5)        117.5 ms 118.1 ms 117.8 ms
```

1. Câte rețele pare să traverseze pachetul? Cum vă dați seama?
2. Între care două salturi traversează traseul Atlanticul? Estimați lungimea acelei legături.

3. Ce ar putea însemna linia * * *? Este traseul întrerupt acolo?
4. La saltul 7, una dintre măsurători este de 61 ms. Dați două explicații.
5. Saltul 9 arată o întârziere puțin mai mică decât saltul 8. Este posibil?

Rezolvare

1. Patru: campusul (salturile 1–2), rețeaua de cercetare (3–4), un furnizor de tranzit (6–8) și furnizorul de conținut (9). Granițele se văd din schimbarea numelui de domeniu (*domain name*) și a blocului de adrese (*address block*).
2. Între salturile 7 și 8: timpul dus-întors sare cu circa 84 ms, deci un sens durează circa 42 ms, adică cel mult 42 × 200 = 8400 km de fibră. Este în acord cu un cablu Frankfurt – New York.
3. Ruterul de la saltul (*hop*) 5 nu a răspuns la sonde (*probes*); multe rutere sunt configurate să nu răspundă sau își limitează frecvența răspunsurilor. Traseul nu este întrerupt, pentru că salturile următoare răspund.
4. O coadă de moment pe traseu sau ruterul însuși, care a generat răspunsul mai încet: răspunsul la sonde este, pentru un ruter, o sarcină de prioritate mică. O măsurătoare lentă din trei este zgomot.
5. Da. Fiecare linie este măsurată cu alte pachete, în alt moment, iar timpul în care un ruter generează un răspuns variază. Doar diferențele mari și consecvente înseamnă ceva.

Temă

- Rezolvați exercițiile 1–5 de la sfârșitul notelor de curs fără să vă uitați la răspunsuri.
- Refaceți Problema 4 pentru utilizatori activi 50 % din timp, cu 15 utilizatori. Mai merită multiplexarea statistică?
- Căutați RFC 867 (*Request for Comments* nr. 867, „Daytime Protocol”), care are o singură pagină, și comparați-l cu proiectul vostru din Problema 6. Ce a lăsat autorul nespecificat?

PARTEA C · SĂPTĂMÂNA 1

Laboratorul 1 (Linux): Explorarea rețelei din linia de comandă

Fișă de laborator de două ore (2 × 50 de minute), asociată Cursului 1

În acest laborator examinați o gazdă reală și Internetul real cu instrumentele standard din Linux. Tot ce ați văzut la curs — interfețe (*interfaces*), legături (*links*), rutere (*routers*), protocoale, întârzieri (*delays*), rețeaua de rețele (*network of networks*) — va apărea pe ecran.

> **La sfârșitul laboratorului ar trebui să puteți**
> • să găsiți interfețele, adresele, poarta implicită (*default gateway*) și serverele DNS ale unei gazde (*host*);
> • să testați accesibilitatea (*reachability*) și să măsurați timpul dus-întors (*round-trip time*, RTT) cu `ping`;
> • să descoperiți ruterele de pe un traseu (*path*) cu `traceroute` și `mtr` și să recunoașteți rețelele traversate;
> • să traduceți nume în adrese (*name resolution*) cu `dig`;
> • să „vorbiți” manual un protocol peste o conexiune TCP cu `nc` și să îl urmăriți cu `curl -v`;
> • să măsurați debitul (*throughput*) și să îl puneți în legătură cu legătura îngustă (*bottleneck*).

Înainte de a începe

- **Mediul de lucru.** Mașina virtuală a cursului (Ubuntu sau Debian), o instalare nativă de Linux sau WSL 2 pe Windows. Toate comenzile se tastează într-un terminal.
- **Pachete software** (*packages*). Dacă lipsește o comandă, instalați o singură dată setul:

```
sudo apt update
sudo apt install -y iproute2 iputils-ping traceroute mtr-tiny dnsutils curl netcat-openbsd tcpdump
```

- **Ajutor.** `man ping`, `ping --help` și `ip addr help`. Citirea paginii de manual face parte din lucrare.
- **Reguli de conduită.** Sondați numai rețeaua laboratorului și servere publice binecunoscute, cu numărul mic de pachete indicat aici. Nu scanați (*scan*) intervale de adrese sau porturi (*ports*) ale unor mașini pe care nu le administrați.
- **Ce predați.** Un raport scurt (un singur fișier, text sau PDF) cu răspunsurile la întrebările numerotate **Q1–Q16** și cu rezultatul comenzilor care susține fiecare răspuns.

Rezultatele tipărite în această fișă sunt ilustrații cu adrese inventate. Ale voastre vor fi diferite.

Plan

Pas	Activitate	Timp
1	Cine sunt eu? Interfețe și adrese	15 min
2	Drumul spre ieșire: rute, vecini, servere DNS	10 min
3	Accesibilitate și timp dus-întors	15 min
4	Traseul: o rețea de rețele	20 min

Pas	Activitate	Timp
	Pauză	10 min
5	Nume și adrese	10 min
6	Un protocol „vorbit” manual	20 min
7	Debitul	10 min
8	Cum arată pachetele (opțional) și încheiere	10 min

Sarcina 1 — Cine sunt eu?

```
hostname
ip -br link
ip -br addr
ip addr show

$ ip -br addr
lo        UNKNOWN   127.0.0.1/8 ::1/128
eth0      UP        10.20.3.57/24 fe80::5054:ff:fe12:3456/64
```

Fiecare linie este o **interfață de rețea** (*network interface*): punctul în care gazda se atașează la o legătură. `lo` este interfața de buclă locală (*loopback*), o legătură software de la gazdă la ea însăși. Opțiunea `-br` dă forma scurtă; fără ea vedeți și adresa hardware (MAC, *Media Access Control address*), după `link/ether`, și MTU (*maximum transmission unit*), adică cel mai mare pachet pe care îl transportă legătura.

- **Q1.** Enumerați interfețele. Care vă conectează la rețeaua laboratorului? Este cu fir sau fără fir?
- **Q2.** Notați adresa ei IPv4 cu lungimea prefixului (*prefix length*; numărul de după bară), adresa MAC și MTU. Cărui nivel al stivei îi aparține fiecare dintre cele două adrese?
- **Q3.** Comparați adresa voastră IPv4 cu a unui coleg. Care parte este aceeași?

Sarcina 2 — Drumul spre ieșire

```
ip route
ip route get 1.1.1.1
ip neigh
cat /etc/resolv.conf
resolvectl status      # pe sistemele cu systemd-resolved

$ ip route
default via 10.20.3.1 dev eth0 proto dhcp metric 100
10.20.3.0/24 dev eth0 proto kernel scope link src 10.20.3.57
```

Aceasta este **tabela de dirijare** (*forwarding table*, numită și *routing table*) a gazdei. A doua linie spune că adresele din propria rețea sunt atinse direct, pe legătură. Linia `default` spune că tot restul este predat unui singur ruter, **poarta implicită** (*default gateway*) — ruterul de margine din curs. `ip neigh` enumeră vecinii (*neighbours*) de pe legătură ale căror adrese MAC au fost aflate de gazdă.

- **Q4.** Care este adresa porții implicite? Câte intrări (*entries*) are tabela voastră de dirijare, față de sutele de mii dintr-un ruter din nucleu? De ce îi ajung unei gazde atât de puține?
- **Q5.** Ce server DNS folosește gazda? Se află în propria voastră rețea?

Sarcina 3 — Accesibilitate și timp dus-întors

`ping` trimite un mic pachet-sondă (*probe*) și așteaptă ecoul (*echo*). Raportează **timpul dus-întors** (*round-trip time*, RTT).

```
ping -c 10 <poarta implicită>
ping -c 10 <serverul web al universității>
ping -c 10 www.kernel.org
ping -c 10 -s 1400 www.kernel.org

64 bytes from 192.0.2.80: icmp_seq=1 ttl=52 time=31.4 ms
...
10 packets transmitted, 10 received, 0% packet loss, time 9012ms
rtt min/avg/max/mdev = 30.9/31.6/33.8/0.8 ms
```

- **Q6.** Completați un tabel cu RTT minim / mediu / maxim și cu pierderile pentru cele trei destinații. Care dintre cele patru componente ale întârzierii explică cea mai mare parte a diferenței dintre ele?
- **Q7.** Lumina parcurge în fibră 200 km pe milisecundă. Din RTT-ul *minim* până la serverul îndepărtat, calculați o limită superioară a distanței până la el. De ce este doar o limită superioară? Se potrivește rezultatul cu locul în care credeți că se află serverul? (Site-urile mari sunt servite din multe locuri; răspunsul vă poate surprinde.)
- **Q8.** Au schimbat pachetele mai mari (`-s 1400`) valoarea RTT? Ce componentă a întârzierii depinde de mărimea pachetului?
- **Q9.** Câmpul `ttl` (*time to live*, „timp de viață”) dintr-un răspuns pornește de la 64, 128 sau 255 la emițător și scade cu unu la fiecare ruter. Estimați câte rutere se află între voi și fiecare destinație.

O gazdă care nu răspunde la `ping` nu este neapărat oprită: multe rețele filtrează aceste sonde. Lipsa răspunsului este o dovadă slabă.

Sarcina 4 — Traseul: o rețea de rețele

```
traceroute -n www.kernel.org
traceroute www.kernel.org
sudo traceroute -I www.kernel.org     # sonde de același tip ca la ping
mtr -rwc 20 www.kernel.org
```

`traceroute` trimite sonde cărora li se permite să traverseze doar 1 ruter, apoi 2, apoi 3 și așa mai departe. Ruterul la care expiră o sondă trimite un mesaj înapoi și își dezvăluie astfel adresa. Cu `-n` vedeți doar adrese; fără această opțiune, instrumentul caută și numele ruterelor, care conțin adesea operatorul și un cod de oraș sau de aeroport. `mtr` repetă măsurătoarea și dă statistici de pierderi (*loss*) și de întârziere pentru fiecare salt (*hop*).

- **Q10.** Câte salturi sunt până la destinație? Se potrivește cu estimarea din Q9?
- **Q11.** Folosind numele ruterelor și blocurile de adrese, marcați locul în care traseul părăsește (a) rețeaua campusului (*campus network*), (b) rețeaua națională de cercetare (*NREN*, *National Research and Education Network*) sau ISP-ul vostru, (c) fiecare rețea următoare. Desenați traseul în stilul Figurii 1 din curs și etichetați fiecare rețea.
- **Q12.** Între care două salturi crește cel mai mult RTT? Ce pas geografic este acela, probabil?
- **Q13.** Urmăriți traseul către un server din România și către unul de pe alt continent. Până la ce salt sunt identice cele două trasee? De ce?

Liniile cu asteriscuri sunt rutere care nu răspund; unele salturi vor arăta o întârziere mai mare decât salturile de după ele. Ambele sunt normale — vedeți Problema 8 de la seminar.

Sarcina 5 — Nume și adrese

```
dig www.kernel.org
dig +short www.kernel.org
dig +short AAAA www.kernel.org
dig -x <adresa unui ruter din Sarcina 4>
```

Oamenii folosesc nume; pachetele poartă adrese. **Sistemul numelor de domenii** (*Domain Name System*, DNS) traduce între ele și este el însuși un protocol, pe care gazda îl rulează cu serverul găsit la Q5. În rezultatul complet al comenzii `dig`, uitați-vă la `ANSWER SECTION` și la linia `Query time` (timpul de interogare — *query time*).

- **Q14.** Ce adrese are numele? Sunt mai multe? Există și una IPv6 (AAAA)? Rulați aceeași interogare de două ori: cum se schimbă timpul de interogare și de ce?

Sarcina 6 — Un protocol „vorbit” manual

Mai întâi urmăriți un program care vorbește HTTP. Liniile care încep cu > sunt trimise de gazda voastră, cele care încep cu < sunt primite.

```
curl -v http://example.org/ -o /dev/null
```

Acum fiți voi browserul. `nc` (netcat) deschide o conexiune (*connection*) TCP și o leagă de tastatură și de ecran. Tastați cele trei linii, apoi apăsați Enter pe o linie goală:

```
nc example.org 80
GET / HTTP/1.1
Host: example.org
Connection: close

```

Dacă serverul închide conexiunea înainte să terminați de tastat, trimiteți cererea dintr-o bucată:

```
printf 'GET / HTTP/1.1\r\nHost: example.org\r\nConnection: close\r\n\r\n' | nc
example.org 80
```

Apoi încălcați intenționat protocolul (*protocol violation*): conectați-vă din nou și tastați `HELLO`, urmat de două Enter. La final, într-un al doilea terminal, uitați-vă la conexiunile deschise ale gazdei cât timp prima este încă activă:

```
ss -tn
```

- **Q15.** Copiați prima linie a răspunsului serverului la cererea corectă și la `HELLO`. Identificați în schimbul vostru cele trei lucruri pe care le definește un protocol: format, ordine, acțiuni (*format, order, actions*). Ce a făcut serverul când formatul a fost încălcat?

Sarcina 7 — Debitul

Descărcați fișierul de test (*test file*) a cărui adresă v-o dă cadrul didactic și lăsați `curl` să raporteze rata medie, în octeți pe secundă, și timpul total:

```
curl -o /dev/null -w 'speed: %{speed_download} B/s   time: %{time_total} s\n' <URL-ul
fișierului de test>
```

- **Q16.** Convertiți rata în Mbit/s. Repetați măsurătoarea de trei ori; apoi repetați-o în timp ce mai mulți colegi descarcă în același moment. Unde credeți că este legătura îngustă: interfața voastră, ieșirea laboratorului spre exterior (*uplink*) sau serverul? Ce dovezi aveți?

Sarcina 8 — Cum arată pachetele (opțional)

Într-un terminal porniți o captură de pachete (*packet capture*, în romgleză „sniffing”) de cinci pachete-sondă; în altul rulați `ping -c 3` către poarta implicită.

```
sudo tcpdump -i any -n -c 5 icmp
sudo tcpdump -i eth0 -n -e -c 5 icmp     # -e afișează și antetul de nivel legătură de date
```

Fiecare linie este un pachet: momentul, adresa sursă, adresa destinație, tipul. Cu `-e` vedeți și adresele MAC — antetul (*header*) de nivel legătură de date care învelește pachetul IP, adică încapsularea făcută vizibilă. Laboratorul de săptămâna viitoare este dedicat capturii de pachete cu Wireshark.

Dacă ceva nu merge

Simptom	Cauza probabilă și ce să încercați
`command not found`	Pachetul software (*package*) nu este instalat; vedeți „Înainte de a începe”.
`ping` merge până la poartă, dar nu mai departe	O problemă de rutare sau de filtrare dincolo de legătura voastră. Verificați `ip route`; încercați altă destinație.
`ping` către o adresă merge, către un nume nu	Rezolvarea numelor nu funcționează. Verificați `/etc/resolv.conf`; încercați `dig @1.1.1.1 example.org`.
`traceroute` arată doar asteriscuri	Sondele implicite sunt filtrate. Încercați `sudo traceroute -I` sau `sudo traceroute -T -p 443`.
`Operation not permitted`	Comanda cere drepturi de administrator (*root*); puneți `sudo` în față.

PARTEA D · SĂPTĂMÂNA 1

Laboratorul 1 (Windows): Explorarea rețelei cu PowerShell

Fișă de laborator de două ore (2 × 50 de minute), asociată Cursului 1

Această fișă acoperă aceleași lucruri ca laboratorul de Linux, pe Windows 10 sau 11. Conceptele sunt identice; instrumentele (*tools*) au alte nume și, pe alocuri, se comportă diferit. Cunoașterea ambelor este o deprindere practică: cele mai multe servere rulează Linux, iar cele mai multe calculatoare de birou pe care vi se va cere să le depanați rulează Windows.

La sfârșitul laboratorului ar trebui să puteți

- să găsiți adaptoarele (*adapters*), adresele, poarta implicită (*default gateway*) și serverele DNS ale unei gazde Windows;
- să măsurați timpul dus-întors (*round-trip time*, RTT) cu `ping` și să urmăriți un traseu (*path*) cu `tracert` și `pathping`;
- să rezolvați nume (*name resolution*) cu `Resolve-DnsName` și să inspectați memoria cache DNS (*DNS cache*) locală;
- să testați un port TCP, să enumerați conexiunile deschise (*open connections*) și să găsiți procesul (*process*) care deține fiecare conexiune;
- să trimiteți manual o cerere HTTP din PowerShell;
- să numiți echivalentul din Linux al fiecărei comenzi folosite.

Înainte de a începe

- **Mediul de lucru.** Deschideți **Windows Terminal** sau **PowerShell**. Nu trebuie instalat nimic. Câteva comenzi, marcate *(admin)*, cer un terminal deschis cu „Run as administrator”.
- **Două familii de instrumente.** Comenzile clasice (`ipconfig`, `ping`, `tracert`, `netstat`) afișează text. Cmdlet-urile PowerShell (*cmdlets*; `Get-NetIPConfiguration`, `Test-NetConnection`) întorc obiecte (*objects*) care pot fi filtrate și sortate. Le veți folosi pe amândouă.
- **O capcană.** În mai vechiul Windows PowerShell 5.1, `curl` este un alias (*alias*) pentru o altă comandă. Tastați întotdeauna `curl.exe` ca să rulați adevăratul curl livrat cu Windows.
- **Ajutor.** `Get-Help Test-NetConnection -Examples` sau `ping /?` pentru comenzile clasice.
- **Regulile de conduită și ce predați** sunt aceleași ca în fișa de Linux: răspunsurile la **Q1–Q16**, cu rezultatele care le susțin. Pentru a salva rezultatul într-un fișier, adăugați la o comandă `| Out-File report.txt -Append`.

Rezultatele tipărite în această fișă sunt ilustrații cu adrese inventate. Ale voastre vor fi diferite.

Plan

Pas	Activitate	Timp
1	Cine sunt eu? Adaptoare și adrese	15 min
2	Drumul spre ieșire: rute, vecini, servere DNS	10 min

Pas	Activitate	Timp
3	Accesibilitate și timp dus-întors	15 min
4	Traseul: o rețea de rețele	20 min
	Pauză	10 min
5	Nume și adrese	10 min
6	Porturi, conexiuni și un protocol „vorbit” manual	20 min
7	Debitul	10 min
8	Linux și Windows față în față; încheiere	10 min

Sarcina 1 — Cine sunt eu?

```
hostname
ipconfig
ipconfig /all
Get-NetAdapter
Get-NetIPAddress -AddressFamily IPv4 | Format-Table InterfaceAlias, IPAddress,
PrefixLength
getmac /v
```

```
Ethernet adapter Ethernet:
   Physical Address. . . . . . . . . : 00-1A-2B-3C-4D-5E
   DHCP Enabled. . . . . . . . . . . : Yes
   IPv4 Address. . . . . . . . . . . : 10.20.3.58(Preferred)
   Subnet Mask . . . . . . . . . . . : 255.255.255.0
   Default Gateway . . . . . . . . . : 10.20.3.1
   DNS Servers . . . . . . . . . . . : 10.20.0.53
```

Windows numește interfețele (*interfaces*) **adaptoare** (*adapters*). `ipconfig /all` enumeră toate adaptoarele, inclusiv pe cele virtuale create de programele de VPN și de virtualizare; `Get-NetAdapter` arată care sunt Up și viteza legăturii lor (*link speed*). „Physical Address” este adresa MAC. Windows scrie prefixul (*prefix*) ca mască de subrețea (*subnet mask*): 255.255.255.0 este același lucru cu /24.

- **Q1.** Enumerați adaptoarele. Care este Up și vă conectează la rețeaua laboratorului? Ce viteză are legătura?
- **Q2.** Notați adresa lui IPv4, masca de subrețea (și lungimea echivalentă a prefixului) și adresa MAC. Cărui nivel îi aparține fiecare dintre cele două adrese?
- **Q3.** Adresa a fost introdusă manual (*static*) sau primită automat („DHCP Enabled”; *Dynamic Host Configuration Protocol*)? De la ce server și până când este valabilă („Lease Expires”)?

Sarcina 2 — Drumul spre ieșire

```
route print -4
Get-NetRoute -DestinationPrefix 0.0.0.0/0
Find-NetRoute -RemoteIPAddress 1.1.1.1
arp -a
Get-NetNeighbor -AddressFamily IPv4
Get-DnsClientServerAddress -AddressFamily IPv4
```

`route print` arată tabela de dirijare (*routing table*) a gazdei. Linia cu destinația de rețea `0.0.0.0` și masca `0.0.0.0` este ruta implicită (*default route*); poarta ei este ruterul vostru de margine (*edge router*). `Find-NetRoute` răspunde la întrebarea „ce rută și ce adresă sursă ar fi folosite pentru a

ajunge la această destinație?”. `arp -a` enumeră vecinii (*neighbours*) de pe legătură și adresele lor MAC.

- **Q4.** Care este adresa porții implicite? Găsiți-i adresa MAC în lista de vecini.
- **Q5.** Ce servere DNS folosește gazda? Se află în propria voastră rețea?

Sarcina 3 — Accesibilitate și timp dus-întors

```
ping -n 10 <poarta implicită>
ping -n 10 <serverul web al universității>
ping -n 10 www.kernel.org
ping -n 10 -l 1400 www.kernel.org
ping -4 -n 4 www.kernel.org          # forțează IPv4

Reply from 192.0.2.80: bytes=32 time=31ms TTL=52
...
Packets: Sent = 10, Received = 10, Lost = 0 (0% loss),
Minimum = 30ms, Maximum = 34ms, Average = 31ms
```

În Windows, `ping` trimite implicit patru sonde (*probes*) (`-n` fixează numărul, `-l` mărimea) și raportează milisecunde întregi, astfel că o poartă aflată pe aceeași legătură apare adesea cu `time<1ms`.

- **Q6.** Completați un tabel cu RTT minim / mediu / maxim și cu pierderile pentru cele trei destinații. Ce componentă a întârzierii explică cea mai mare parte a diferenței?
- **Q7.** Din RTT-ul minim până la serverul îndepărtat, calculați o limită superioară a distanței până la el (200 km pe milisecundă în fibră; nu uitați că RTT înseamnă dus *și întors*). Se potrivește cu locul în care credeți că se află serverul?
- **Q8.** Au schimbat pachetele mai mari (`-l 1400`) valoarea RTT? Ce componentă a întârzierii depinde de mărimea pachetului?
- **Q9.** Cereți unui coleg adresa mașinii lui Windows și dați-i ping. Dacă nu vine niciun răspuns, este mașina oprită? (Windows Firewall — paravanul de protecție, *firewall* —, implicit, nu răspunde la aceste sonde venite de la alte mașini.) Ce vă învață aceasta despre interpretarea unui ping eșuat?

Sarcina 4 — Traseul: o rețea de rețele

```
tracert -d www.kernel.org
tracert www.kernel.org
Test-NetConnection www.kernel.org -TraceRoute
pathping -n -q 10 www.kernel.org
```

`tracert` funcționează ca `traceroute` din Linux: sonde care pot traversa doar 1, 2, 3… rutere, fiecare ruter la care expiră o sondă trimițând un mesaj înapoi. Fiecare ruter traversat este un salt (*hop*). Cu `-d` numele nu mai sunt căutate, ceea ce este mai rapid. `pathping` urmărește mai întâi traseul și apoi măsoară pierderile (*loss*) la fiecare salt; durează un minut sau două, așa că porniți-l și citiți mai departe.

O deosebire contează: `tracert` trimite același tip de sondă ca `ping` (ICMP echo), pe când `traceroute` din Linux trimite implicit pachete UDP (*User Datagram Protocol*). Rețelele pot filtra un tip și nu pe celălalt, astfel că cele două instrumente pot arăta rezultate diferite pe același traseu.

- **Q10.** Câte salturi sunt până la destinație? Comparați cu TTL-ul (*time to live*) văzut în Sarcina 3: Linux și echipamentele de rețea pornesc de obicei de la 64 sau 255, Windows de la 128.

- **Q11.** Folosind numele ruterelor și blocurile de adrese, marcați locul în care traseul părăsește rețeaua campusului (*campus network*), rețeaua națională de cercetare (*NREN*) sau ISP-ul vostru și fiecare rețea următoare. Desenați traseul în stilul Figurii 1 din curs.
- **Q12.** Între care două salturi crește cel mai mult RTT? Ce pas geografic este acela, probabil?
- **Q13.** În rezultatul `pathping`, arată vreun salt pierderi? Dacă un salt intermediar arată pierderi, dar destinația finală nu, există o problemă reală?

Sarcina 5 — Nume și adrese

```
Resolve-DnsName www.kernel.org
Resolve-DnsName www.kernel.org -Type AAAA
Resolve-DnsName <adresa unui ruter din Sarcina 4>      # căutare inversă
nslookup www.kernel.org
ipconfig /displaydns | Select-Object -First 40
Clear-DnsClientCache
```

`Resolve-DnsName` este corespondentul PowerShell al comenzii `dig` din Linux; `nslookup` este instrumentul mai vechi, prezent pe orice sistem. Windows păstrează răspunsurile recente într-o **memorie cache DNS** (*DNS cache*) locală, pe care o afișează `ipconfig /displaydns`.

- **Q14.** Ce adrese are numele? Există și una IPv6? Uitați-vă la coloana TTL (*time to live*): cât timp poate gazda să refolosească răspunsul fără să mai întrebe? Găsiți numele în cache, goliți cache-ul și verificați că a dispărut.

Sarcina 6 — Porturi, conexiuni și un protocol „vorbit” manual

La un program server se ajunge printr-o adresă *și* un număr de **port** (*port number*); serverele web ascultă pe portul 80 (HTTP) și pe 443 (HTTPS). Testați dacă se poate deschide o conexiune (*connection*) TCP:

```
Test-NetConnection example.org -Port 80
Test-NetConnection example.org -Port 81
```

Urmăriți un program care vorbește HTTP (> marchează liniile trimise, < pe cele primite):

```
curl.exe -v http://example.org/ -o NUL
```

Acum fiți voi browserul. Windows nu are `nc` (netcat), așa că deschideți conexiunea TCP din PowerShell și scrieți cererea în ea. Lipiți blocul în întregime:

```
$client = New-Object System.Net.Sockets.TcpClient('example.org', 80)
$stream = $client.GetStream()
$writer = New-Object System.IO.StreamWriter($stream)
$writer.NewLine = "`r`n"; $writer.AutoFlush = $true
$reader = New-Object System.IO.StreamReader($stream)
$writer.WriteLine('GET / HTTP/1.1')
$writer.WriteLine('Host: example.org')
$writer.WriteLine('Connection: close')
$writer.WriteLine('')
$reader.ReadToEnd()
$client.Close()
```

Rulați-l din nou, cu prima linie a cererii înlocuită cu `HELLO`. Apoi enumerați conexiunile deschise ale mașinii și aflați ce program (proces — *process*, identificat prin *PID*) deține una dintre ele:

```
netstat -ano | Select-String ESTABLISHED
Get-NetTCPConnection -State Established | Sort-Object RemotePort | Format-Table
LocalPort, RemoteAddress, RemotePort, OwningProcess
Get-Process -Id <un număr din coloana OwningProcess>
```

- **Q15.** Copiați prima linie a răspunsului serverului la cererea corectă și la HELLO. Identificați în schimbul vostru formatul, ordinea și acțiunile definite de protocol. Ce s-a întâmplat cu portul 81 și cât a durat până când testul a renunțat? De ce atât de mult?

Sarcina 7 — Debitul

```
curl.exe -o NUL -w "speed: %{speed_download} B/s   time: %{time_total} s\n" <URL-ul fișierului de test>
```

Cât timp rulează descărcarea, deschideți **Task Manager → Performance** și selectați adaptorul, ca să urmăriți în timp real rata de recepție.

- **Q16.** Convertiți rata (*rate*) în Mbit/s și comparați-o cu viteza legăturii de la Q1. Repetați de trei ori, apoi în timp ce mai mulți colegi descarcă împreună. Unde este legătura îngustă (*bottleneck*): adaptorul vostru, ieșirea laboratorului spre exterior (*uplink*) sau serverul? Ce dovezi aveți?

Sarcina 8 — Linux și Windows față în față

Completați coloanele de Windows din memorie, înainte de a vă uita înapoi; apoi păstrați tabelul.

Scop	Linux	Windows (clasic)	Windows (PowerShell)
Interfețe și adrese	`ip addr`	`ipconfig /all`	`Get-NetIPConfiguration`
Tabela de dirijare	`ip route`	`route print`	`Get-NetRoute`
Ruta către o destinație	`ip route get`	–	`Find-NetRoute`
Vecinii de pe legătură	`ip neigh`	`arp -a`	`Get-NetNeighbor`
Accesibilitate, RTT	`ping -c`	`ping -n`	`Test-Connection`
Traseu	`traceroute`, `mtr`	`tracert`, `pathping`	`Test-NetConnection -TraceRoute`
Căutare de nume	`dig`	`nslookup`	`Resolve-DnsName`
Testarea unui port TCP	`nc -vz host port`	–	`Test-NetConnection -Port`
Conexiuni deschise	`ss -tn`	`netstat -ano`	`Get-NetTCPConnection`
Client HTTP	`curl -v`	`curl.exe -v`	`Invoke-WebRequest`
Captură de pachete	`tcpdump`, Wireshark	`pktmon` *(admin)*, Wireshark	–

Dacă este instalat Windows Subsystem for Linux, toate comenzile din fișa de Linux rulează și pe această mașină, în WSL. Încercați acolo `ip addr` și comparați cu `ipconfig`: adresele diferă, pentru că WSL 2 stă implicit în propria rețea virtuală (*virtual network*), în spatele gazdei Windows.

Dacă ceva nu merge

Simptom	Cauza probabilă și ce să încercați
`Request timed out` către orice destinație	Nu există conectivitate (*connectivity*) sau sondele sunt filtrate (*filtered*). Verificați în `ipconfig` că aveți adresă și poartă; dați ping mai întâi porții.
Adresa începe cu 169.254	Gazda nu a primit automat o adresă. Verificați cablul sau Wi-Fi; rulați `ipconfig /renew`.

Simptom	Cauza probabilă și ce să încercați
Ping către o adresă merge, către un nume nu	Rezolvarea numelor nu funcționează. Verificați Q5; încercați `Resolve-DnsName example.org -Server 1.1.1.1`.
`curl -v` afișează un obiect HTML și nicio urmă a schimbului	Ați rulat aliasul din PowerShell. Tastați `curl.exe`.
`Access is denied` sau „requires elevation”	Deschideți terminalul cu „Run as administrator”.

Săptămâna viitoare

Seminarul 2 tratează, cu alte probleme, întârzierea de așteptare în coadă și modelele de referință. Laboratorul 2, pe ambele sisteme, este captura de pachete (*packet capture*, în romgleză „sniffing”) cu Wireshark: veți vedea antetele din Figura 3 în pachete reale.